SIMATS ENGINEERING

Short Answer Test

Name: M.S.Charishma

Reg.no:192311393

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3

5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3
7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3
8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3

13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3
19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3

22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3

Code of Conduct:

I __M.S.Charishma___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.S.Charishma

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

ANSWERS

1. Testing as an Engineering Activity

Random testing gives inconsistent coverage because there is no fixed plan. Testing should be treated as an engineering activity: study requirements, identify risks, design test cases, execute them systematically and record defects.

For a login module, tests should cover valid credentials, wrong passwords, blank fields, locked accounts and repeated attempts. Test cases should be reviewed and linked to requirements.

Using planned test design and defect tracking makes results repeatable and improves defect detection. It also prevents important conditions from being missed simply because the tester did not think of them during random execution.

2. Defined Testing Process for a Banking System

A banking system needs a defined process because transaction failures can affect customers directly. Testing should begin with requirement review, followed by test planning, test design, execution, defect reporting, retesting and regression testing.

For fund transfer, test successful transfers, insufficient balance, invalid account details, limits and network interruption. When a defect is corrected, the original case and related transaction cases should be run again.

A standard process gives every release the same basic quality checks, makes defects easier to track and reduces the chance of old problems returning.

3. Structured Testing in SDLC

Testing should continue throughout the SDLC rather than starting only after coding. During requirements, testers check completeness and testability. During design they identify risks; during development they prepare unit and integration tests; later they perform system and acceptance testing.

For an e-commerce price rule, the requirement can be reviewed first, the calculation tested in its module, then checked again when cart and payment modules interact.

Process discipline catches problems earlier and reduces integration and system-level defects.

4. Error, Defect and Failure

An error is a human mistake, a defect is the fault introduced into the software because of that mistake, and a failure is the incorrect behaviour observed during execution.

For example, a developer may misunderstand the password rule. That misunderstanding is the error. If the program accepts a five-character password, the incorrect validation is the defect. When the application actually accepts it, the tester observes a failure.

Keeping these terms separate makes defect reports clearer and helps the team understand whether the problem started in requirements, coding or execution.

5. Early Defect Detection

The principle of early testing should be applied. Defects found during requirements or design review are normally cheaper to correct than defects found after release.

Testers can review requirements, identify ambiguous conditions and prepare test ideas before coding is finished. Static reviews can also reveal mistakes without executing the program.

For example, if a payment timeout is not defined, raising the issue during requirement analysis prevents developers from making different assumptions. Early detection reduces rework and debugging effort.

6. Tester in Requirement Analysis

A tester can contribute before coding by checking whether requirements are complete, consistent, clear and testable. The tester should question missing rules and unusual conditions.

In a healthcare system, useful questions include what happens when patient information is missing, who may modify records, and what happens when two users update the same record.

Finding such ambiguity early prevents different interpretations during development. Early tester involvement therefore improves quality by preventing requirement defects instead of only finding their effects later.

7. Unclear Requirements in Payroll

Defects can originate before programming when requirements are incomplete or ambiguous. Developers may implement different interpretations of the same business rule.

Suppose overtime starts after eight hours according to one interpretation but after nine hours according to another. The payroll program may calculate salary incorrectly even though its code follows the chosen rule.

Requirement reviews, examples, acceptance criteria and traceability can prevent this. Clear requirements give developers and testers the same understanding and reduce defects at the source.

8. Boundary Input Defect

A crash occurring at a minimum or maximum input is a boundary-related functional defect. It shows that the program does not handle a limit correctly.

Boundary Value Analysis focuses on values at and around limits. If an amount is allowed from ₹100 to ₹10,000, useful tests include ₹100, ₹101, ₹9,999, ₹10,000 and nearby invalid values.

Testing only ordinary values such as ₹1,000 may miss the problem. Boundary-focused cases could have exposed the defect before deployment.

9. Defect Repository and Test Design

A defect repository should be used as a source of testing knowledge, not just as storage. Historical records contain information about affected modules, causes, severity and previous failures.

If discount calculation has failed in earlier releases, those cases should be retained in regression testing and new tests should cover related percentages, limits and invalid combinations.

Reviewing repository data helps testers concentrate on weak areas, avoid repeating old mistakes and design more relevant test cases for future releases.

10. Functional Defect in ATM Withdrawal

Incorrect transaction logic in an ATM is a functional defect because the system does not perform the required business operation correctly. For example, the balance may be deducted without cash being dispensed.

Withdrawal testing should include valid amounts, insufficient balance, invalid amounts, daily limits and interrupted transactions. Both the cash output and account balance must be checked.

Requirement-based test cases and regression testing after fixes could have detected the problem earlier. High-risk banking transactions should never depend only on random tests.

11. Defect Clustering

Defect clustering means that many defects are often concentrated in a small number of modules. This observation can be used to prioritize testing effort.

Past defect records can identify modules with frequent or severe failures. Such modules should receive additional test cases, reviews and regression coverage.

For example, payment and authentication may deserve more attention than a simple information page. Prioritizing high-risk modules uses limited testing time effectively and increases the chance of finding important defects early.

12. Performance Defect

Slow response under heavy user load is mainly a performance defect. The software may function correctly with a few users but fail to meet response-time requirements when demand increases.

Possible causes include inefficient database queries, limited server resources or poor handling of concurrent requests. Load and stress testing can reproduce the problem.

Correct classification matters because ordinary functional testing may not expose it. Once the bottleneck is identified, the team can optimize the relevant component and repeat the performance test.

13. Historical Data for Regression

Historical defect data can be reused when preparing regression tests. Earlier high-severity and repeated defects should remain in the test suite, especially after changes to the same module.

For example, if a tax calculation failed for a certain salary range in an earlier release, that exact condition and nearby cases should be retested.

This approach uses previous experience to strengthen current coverage. It saves time because testers do not need to rediscover important scenarios from the beginning and reduces the chance of old defects returning.

14. Tester Role and Collaboration

Testers help reduce integration defects by reviewing requirements, discussing interfaces and clarifying expected behaviour before modules are combined.

If an order module sends an address to a delivery module, the tester can ask early what should happen when the address is incomplete or changed. This prevents developers and testers from making different assumptions.

Regular requirement discussions, shared acceptance criteria and evidence-based defect communication improve collaboration. Early involvement reduces misunderstandings and makes integration testing smoother.

15. Defect Prevention

Repeated defects need prevention, not only repeated fixing. The team should investigate the common root cause and improve the process that allowed it.

Useful actions include root-cause analysis, requirement and design reviews, coding reviews, better validation rules and adding important previous defects to regression testing.

For example, if several screens repeatedly accept invalid input, a common validation rule and reusable test cases may be introduced. Prevention reduces rework and gradually improves software quality across releases.

16. Standardized Mobile Testing

Inconsistent testing means different testers may give different levels of coverage. A common strategy should define test-case formats, review rules, execution practices, defect reporting and regression criteria.

Mobile testing should also consider different devices, screen sizes, network changes, interruptions, restarts and invalid user actions.

For a payment feature, test not only a normal Wi-Fi connection but also a connection that becomes unstable during the transaction. Standardized practices make testing repeatable and improve reliability.

17. Testing and Debugging

Testing is performed to find defects and evaluate whether software meets its requirements. Debugging starts after a problem is identified and focuses on locating and correcting its cause.

For an ATM balance-update failure, the tester reproduces and reports the issue. The developer then traces the transaction code and fixes the incorrect database update. The tester retests the fix and performs related regression checks.

Thus testing provides evidence of a problem, while debugging investigates and removes its cause. Both activities are necessary but they are not the same.

18. Cross-Browser Testing

A cross-browser strategy should first identify browsers, versions and operating systems relevant to the target users. Important shopping functions such as login, search, cart, checkout and payment should be included.

Browsers can interpret JavaScript, CSS, fonts and responsive layouts differently. Developer tools and cross-browser testing environments can help reproduce such differences.

After a browser-specific defect is fixed, the affected scenario should be retested and related pages should undergo regression testing. A planned browser matrix gives customers a more consistent experience.

19. Validation Gaps in Logistics

Validation should confirm that correct data is accepted and unsuitable data is rejected. The team should examine whether blank, invalid, boundary and incorrectly formatted inputs were missed.

For a delivery-weight field, test valid values, zero, negative numbers, very large values and non-numeric input when applicable. The exact input and output should be recorded in any defect report.

Systematic validation prevents incorrect data from reaching later processing stages and reduces inconsistent results between modules.

20. Systematic Test Case Design

Random testing may find some defects, but it cannot show whether important requirements are covered. A systematic approach begins with requirements and identifies inputs, conditions and expected results.

Equivalence partitioning, boundary value analysis, negative testing and decision-based techniques can be selected according to the problem. For an age range of 18 to 60, test values inside the range and at both boundaries and outside them.

Each test then has a clear purpose, coverage becomes easier to review and reported defects are easier to reproduce.

21. Healthcare Data Consistency

Integration problems can occur when modules use different fields, update rules or synchronization methods. To find the root cause, controlled patient data should be followed through registration, appointments, billing and other connected modules.

After changing a shared value, verify that every dependent module shows the expected value. Also test simultaneous updates and incomplete transactions where relevant.

For example, a changed contact number should appear consistently wherever the system is designed to share it. Integration-focused testing improves confidence in data consistency.

22. Testers Across SDLC

Testers have useful roles throughout the SDLC. They review requirements for ambiguity, identify risks during design, prepare test conditions during development and verify complete behaviour during system and acceptance testing.

Early involvement is important because a requirement mistake discovered after coding can require much more rework. Regular developer-tester discussions and shared acceptance criteria can prevent such delays.

The tester therefore contributes to prevention as well as detection, helping the team build quality into the product from the beginning.

23. Peak Usage and Coverage

A system that works for normal usage may still fail when many users access it at the same time. Functional testing alone does not prove capacity.

Test planning should include realistic concurrent users, sudden increases in traffic and long-duration usage where required. For an online learning system, many students can log in, open lessons and submit assignments together while response time and errors are measured.

Testing these scenarios before release exposes capacity problems early and helps the team improve reliability under real usage conditions.

24. Defect Documentation

A defect record should contain enough information for another person to reproduce the problem. Useful details include title, module, preconditions, steps, expected result, actual result, severity, priority, status and resolution.

Good documentation prevents repeated investigation and preserves knowledge between releases. The repository can also be searched for repeated defects and modules with high failure rates.

For example, repeated payment failures can be grouped and used to strengthen the payment regression suite. Proper documentation therefore improves both tracking and analysis.

25. Banking Transaction Failures

The banking system should use a planned, risk-based testing strategy. Critical functions such as fund transfer, payment, balance update and transaction history need strong coverage.

Important cases include successful transactions, insufficient balance, invalid accounts, duplicate requests, network interruption and session expiry. A delayed transfer button should also be tested to ensure one click cannot create two transactions.

Fixed defects should be retested and related functions should undergo regression testing. This structured approach gives much stronger reliability than random execution.

26. Edge Case Testing

Normal inputs alone are not enough because failures often occur at limits and unusual conditions. Test design should deliberately include minimum, maximum, empty, zero, duplicate and invalid cases where applicable.

For a system allowing 1 to 100 items, test 1, 2, 99, 100 and nearby invalid values such as 0 and 101.

Boundary testing is useful because it targets the points where validation and calculation logic commonly changes. Including these cases improves defect detection before deployment.

27. Process-Based Testing

A common testing process should be followed across modules: requirement review, test design, review of test cases, execution, defect reporting, retesting and regression.

The test cases themselves will differ by module. Payment may require transaction scenarios while profile management needs validation checks. However, both can use the same documentation and defect-reporting rules.

Templates, checklists, peer reviews and periodic result reviews help maintain consistency. Standardization reduces uneven quality without forcing every module to be tested identically.

28. Unexpected Input Validation

The missing area is likely negative testing. The application should be checked with blank values, incorrect formats, wrong data types, values outside permitted limits and unexpected combinations.

For a quantity field, test a normal value such as 5, but also 0, negative values, very large values and invalid text according to the requirement.

Good validation rejects unsuitable input with a clear response instead of crashing or producing incorrect output. This makes the application more robust when users do not follow the expected input pattern.

29. Reducing Defect Leakage

Frequent production defects may be caused by incomplete requirements, weak coverage, insufficient regression testing or failure to test realistic conditions. Production defects should be reviewed to find the reason they escaped.

If a production failure occurs at a transaction limit, that condition should be added to the regression suite and the original requirement and test design should be reviewed.

Risk-based testing, realistic test data, regression testing and release-readiness reviews can reduce leakage. The goal is not only to fix escaped defects but also to learn from them.

30. Early Defect Detection

The strategy should begin testing activities early and continue throughout development. Requirements and design should be reviewed, high-risk areas identified and test conditions prepared before the final build.

Reviews, unit and integration tests, system testing, defect tracking and regression testing all contribute to early detection.

For example, if a bank-charge rule is unclear, resolving it during requirement review is much cheaper than correcting the misunderstanding after the billing module is completed. Early detection reduces rework, debugging effort and overall project cost.